

RTL Design Sherpa

RTL AMBA CXS Module Reference — Rev 0.90

October 2026

Table of Contents

1 CXS (Credited eXtensible Stream) Modules....5
1.1 Overview....5
1.2 Module Categories....6
1.3 Properties and Parameters....7
1.3.1 Configuration checks....9
1.4 Streams and Interleaving....9
1.5 Credits Beyond the Spec....10
1.6 CXSCNTL Layout....10
1.7 Link Control (CXS_LINK_CTRL = 1)....11
1.8 Implemented Signal Set....11
1.9 Verification....12
1.10 References....12
1.11 Navigation....13
2 CXS Master....14
2.1 Overview....14
2.1.1 Key Features....14
2.2 Parameters....14
2.2.1 Derived Values (do not override)....16
2.3 Ports....16
2.3.1 Clock, Reset and Configuration....16
2.3.2 FUB Interface (valid/ready, input side)....17
2.3.3 CXS Transmitter Interface....17

2.3.4 Check Signals (CXS_CHECK = 1)....18
2.3.5 Status....18
2.4 Functional Description....19
2.4.1 Credits....19
2.4.2 Link control....20
2.4.3 Packet tracking....20
2.5 Timing Characteristics....20
2.6 Usage Example....21
2.7 Design Notes....21
2.8 Related Modules....22
2.9 Testing....22
2.10 References....22
2.11 Navigation....22
3 CXS Master with Clock Gating....23
3.1 Overview....23
3.2 When It Gates....23
3.3 Activity Terms....23
3.4 Testing....24
3.5 Navigation....24
4 CXS Slave....25
4.1 Overview....25
4.1.1 Key Features....25
4.2 Parameters....25
4.3 Ports....26

4.3.1 Clock, Reset and Configuration....26
4.3.2 CXS Receiver Interface....27
4.3.3 Check Signals (CXS_CHECK = 1)....27
4.3.4 FUB Interface (valid/ready, output side)....27
4.3.5 Status....28
4.4 Functional Description....28
4.4.1 Credit pool....28
4.4.2 Link control....28
4.5 Timing Characteristics....29
4.6 Usage Example....29
4.7 Related Modules....30
4.8 Testing....30
4.9 References....30
4.10 Navigation....31
5 CXS Slave with Clock Gating....32
5.1 Overview....32
5.2 When It Gates....32
5.3 Activity Terms....32
5.4 Testing....33
5.5 Navigation....33

1 CXS (Credited eXtensible Stream) Modules

Location: `rtl/amba/cxs/` **Specification:** Arm *AMBA CXS Protocol Specification*, ARM IHI 0079, Issues A, B, C and D **Status:** New (2026-10-15); GATE/FUNC/FULL green

1.1 Overview

CXS is Arm's point-to-point packet stream protocol for wide links. Its main use is between an on-chip interconnect and a PCIe controller carrying CCIX or CXL. Data moves in one direction, one *flit* (one cycle of `CXSDATA`) at a time. A flit can hold several packets, and a packet can span several flits.

CXS differs from AXI-Stream in one fundamental way: **there is no READY.** The receiver grants credits on `CXSCRDGNT`, one per cycle. The transmitter spends one credit per flit, and with link control it can hand a credit back on `CXSCRDRTN`. Every credit the receiver grants is a promise of buffer space for one flit, so the receiver is never back-pressured on the wire.

This directory holds one family covering **every published issue**. Each issue is a strict superset of the one before it. An issue is therefore a set of legal property values, not a different protocol, and the `CXS_ISSUE` parameter selects which set is checked at elaboration.

Issue	Date	What it adds
A	Mar 2018	First release: `CXSDATAFLITWIDTH` 256/512/1024, `CXSMAXPKTPERFLIT` 2..4, up to 15 credits, `CXSCHECKTYPE`, `CXSLINKCONTROL`
B	Dec 2020	Multiple protocol streams: `CXSLAST` (`CXS_LAST`), `CXSPRCLTYPE` (`CXS_PROTOCOL_TYPE`), and their check signals
C	Dec 2023	One packet per flit (`CXSMAXPKTPERFLIT` = 1, no `CXSCNTL`, any width 8..2048), `CXS_MAX_CREDIT` up to 63, clock and reset

Issue	Date	What it adds
		defined
D	Sep 2025	2048-bit flits with several packets, `CXS_START_ALIGNMENT` = 4, `CXSSRCID`/`CXSTGTID` (0..8 bits), `CXSUSER` (0..128 bits), CXS-Lite named

1.2 Module Categories

Module	Role	Page
`cxs_master`	Transmitter: FUB valid/ready flits in, credited CXS link out	cxs_master.md
`cxs_slave`	Receiver: credited CXS link in, FUB valid/ready flits out; the receive FIFO is the credit pool	cxs_slave.md
`cxs_master_cg`	`cxs_master` behind `amba_clock_gate_ctrl`; gates only in link STOP	cxs_master_cg.md
`cxs_slave_cg`	`cxs_slave` behind `amba_clock_gate_ctrl`; a rising `CXSACTIVEREQ` wakes it	cxs_slave_cg.md
`cxs_pkg` (`rtl/amba/includes/`)	Widths, `CXSCNTL` field offsets, per-issue legality (`config_error`)	this page
`cxs_cntl_track`	Packet-open tracking and `CXSCNTL` format checks, shared by both endpoints	this page
`cxs_odd_byte_parity`	`Odd_Byte_Parity` generate/check for one signal of any width	this page

Module	Role	Page
cxs_stream_key	The tracked part of a flit's stream identity: {PRCLTYPE, SRCID, TGTID} low bits	this page
cxs_link_snoop	Passive model of a link from its wires alone: credits in flight, link state, credit/link/per-stream framing checks (feeds cxs_intf_observer)	this page
cxs_master_monlite / cxs_slave_monlite	The endpoints with cxs_monitor_lite tapping their FUB handshake and status; packets tagged PROTOCOL_CXS	cxs_monitor_lite.md
axis_to_cxs / cxs_to_axis	AXI-Stream beats to CXS flits and back (one packet per TLAST run, TSTRB carried in CXSUSER). They live in projects/components/utility-ip/converters/rtl/	Converters MAS 3.17 (17_axis_to_cxs.md)

The monitor wrappers have no _cg variants (clock gating is an FPGA-only concern here). Both endpoints export credit_starved for the monitor: on the transmitter, a flit is waiting in RUN with no credit; on the receiver, no credit can be granted because the FIFO holds every slot.

1.3 Properties and Parameters

Every endpoint takes the same property set. Parameter names follow the spec property they implement.

Spec property	Parameter	Values	Issue
(issue)	CXS_ISSUE	1..4 = A..D (cxs_pkg::CXS_ISSUE_A..D)	-

Spec property	Parameter	Values	Issue
CXSDATAFLITWIDTH	CXS_DATA_WIDTH	256/512/1024; 2048 with several packets (D); 8..2048 by 8 with one packet (C)	A
CXSMAXPKTPERFLIT	CXS_MAX_PKT_PER_FLIT	2..4; 1 from C	A
CXS_START_ALIGNMENT	CXS_START_ALIGN	16; 4 from D	D
CXS_LAST	CXS_HAS_LAST	0/1	B
CXS_PROTOCOL_TYPE	CXS_HAS_PRCLTYPE	0/1	B
CXS_SRCID_WIDTH / CXS_TGTID_WIDTH	same names	0..8	D
CXS_USER_WIDTH	same name	0..128	D
CXS_MAX_CREDIT	same name	1..15; 1..63 from C	A
CXSLINKCONTROL	CXS_LINK_CTRL	1 = Explicit_Credit_Return, 0 = None	A
CXSCHECKTYPE	CXS_CHECK	1 = Odd_Byte_Parity, 0 = None	A
(beyond spec)	CXS_CREDIT_BEYOND_SPEC	1 = CXS_MAX_CREDIT may exceed the cap (rule 12 skipped)	-
(stream tracking)	CXS_STREAM_PRCL_BITS / CXS_STREAM_SRCID_BITS / CXS_STREAM_TGTID_BITS	low bits of each field that tell interleaved streams apart; 0/0/0 = one stream	B

CXSCONTINUOUSDATA and CXSERRORFULLPKT constrain what the *source of packets* does, so they are not parameters of these endpoints. Both endpoints carry flits unchanged. The transmitter never deactivates the link while a packet is open, which CXSCONTINUOUSDATA requires.

1.3.1 Configuration checks

`cxs_pkg::config_error()` returns 0 for a legal property set, or the number of the first rule broken. Each endpoint calls it from an elaboration-time `param_check` and stops with `$fatal` naming the rule. The framework's `CXSConfig.config_error()` implements the same rules in the same order, so a rule number means the same thing on both sides.

Rule	Requirement
1	Issue is A..D
2	`CXSMAXPKTPERFLIT` is 1..4
3	One packet per flit needs Issue C or later
4	With one packet per flit, the width is 8..2048 and a multiple of 8
5	With one packet per flit, `CXS_LAST` and `CXS_PROTOCOL_TYPE` are false
6	With several packets per flit, the width is 256/512/1024, plus 2048 from Issue D
7	A 256-bit flit carries at most 2 packets, unless start alignment is 4 (Issue D)
8	Start alignment is 4 or 16
9	Start alignment 4 needs Issue D
10	`CXSLAST` / `CXSPRCLTYPE` need Issue B or later
11	`CXS_MAX_CREDIT` is at least 1
12	`CXS_MAX_CREDIT` is at most 15 before Issue C, 63 from Issue C (skipped with `CXS_CREDIT_BEYOND_SPEC`)
13	Source/target ID widths are 0..8, user width 0..128
14	`CXSSRCID` / `CXSTGTID` / `CXSUSER` need Issue D

1.4 Streams and Interleaving

A CXS stream is the flits that share `CXSPRCLTYPE`, `CXSSRCID` and `CXSTGTID` (IHI 0079D 2.5.2). Flits of different streams may interleave (with `CXSCONTINUOUSDATA`, only where `CXSLAST` allows), so

packet framing is legal or not per stream: a flit may carry a START for stream B while stream A still has a packet open.

`cxs_cntl_track` keeps one packet-open bit per stream, selected by a key that `cxs_stream_key` builds from the low `CXS_STREAM_*_BITS` of the three fields. Tracking every possible stream would cost 2^19 bits, so the key bounds the state at 2^(P+S+T) bits. Streams that differ only above the key share one bit, so the key must cover every stream that can have a packet open at once. RAPIDS keys on the channel bits of `CXSSRCID`. The defaults (0/0/0) track the whole link as one stream, which is exactly the behaviour before 2026-10-16. Both ends of a link, and any monitor or snoop on it, must use the same key. With the key off, interleaved streams set `protocol_error`; `val/amba/test_cxs_stream_interleave.py` asserts both halves.

The framework's `CXSDepacketizer` and `CXSMonitor` track open packets per stream on the full spec key, so they need no parameter.

1.5 Credits Beyond the Spec

`CXS_MAX_CREDIT` is capped at 63 (15 before Issue C). `CXS_CREDIT_BEYOND_SPEC = 1` lifts the cap and nothing else: rule 12 is skipped, the credit counters widen to hold the pool, and every other rule still applies. It exists for a receiver whose buffer is a whole SRAM (the RAPIDS CXS tops default the pool to one channel's SRAM). Both ends of the link must be built with it; the framework's `CXSConfig(credit_beyond_spec=True)` mirrors it.

1.6 CXSCNTL Layout

`CXSCNTL` marks where packets start and end inside a flit. With N = `CXSMAXPKTPERFLIT`, the fields from bit 0 upward are:

```
START[N-1:0] | START0PTR .. START(N-1)PTR | END[N-1:0] | ENDERROR[N-1:0] | END0PTR .. END(N-1)PTR
```

- A start pointer is log2(W/128) bits with 16-byte alignment, or log2(W/32) with 4-byte alignment. The first byte of a packet is `STARTnPTR << 4` (or `<< 2`).
- An end pointer is always log2(W/32) bits and points at the packet's last aligned 4 bytes, so its last byte is `(ENDnPTR << 2) + 3`.
- `START` and `END` are thermometer codes, and `ENDERROR[n]` requires `END[n]`.

So `CXSCNTLWIDTH = N * (3 + start_ptr_w + end_ptr_w)`, and it is 0 when N = 1. `cxs_pkg::cntl_width()` computes it. The framework unit test `tests/unit/test_cxs_config.py` checks the formula against all 22 rows of IHI 0079D Tables 4-5 and 4-7, including each END field's bit position.

1.7 Link Control (CXS_LINK_CTRL = 1)

Two signals, CXSACTIVEREQ (driven by the transmitter) and CXSACTIVEACK (driven by the receiver), form a four-phase handshake. The link state is simply the pair of signal values; there is no separate state machine.

{REQ, ACK}	State	Flits	Credits
00	STOP	none	all held by the receiver
10	ACTIVATE	none	the transmitter must accept early grants
11	RUN	yes	granted, spent, optionally returned
01	DEACTIVATE	in-flight only	the transmitter returns every credit; ACK falls only when all are home

CXSDEACTHINT lets the receiver ask the transmitter to take the link down. Only the transmitter starts a state change. CXSACTIVEREQ may be captured asynchronously, so cxs_slave synchronizes it (ACTIVEREQ_SYNC_STAGES).

1.8 Implemented Signal Set

Every signal of IHI 0079D Table 3-1 and every check signal of Table 3-2 is implemented. An absent optional signal keeps a one-bit port, tied low on outputs and ignored on inputs, the same convention the AXIS endpoints use.

Signal	Present when
CXSVALID, CXSDATA, CXSCRDGNT	always
CXSCNTL	CXS_MAX_PKT_PER_FLIT > 1
CXSLAST / CXSPRCLTYPE	CXS_HAS_LAST / CXS_HAS_PRCLTYPE
CXSSRCID / CXSTGTID / CXSUSER	width > 0
CXSCRDRTN, CXSACTIVEREQ, CXSACTIVEACK, CXSDEACTHINT	CXS_LINK_CTRL = 1
*CHK (one per signal above, except	CXS_CHECK = 1 and the signal is present

Signal	Present when
CXSDEACTHINT)	

1.9 Verification

The BFMs are the `cxs` family of the RDS-DV framework (`CocoTBFramework.components.cxs`): `CXSTransmitter`, `CXSReceiver`, `CXSMonitor`, `CXSConfig`, `CXSPacketizer` and `CXSDepacketizer`. The FUB side of each endpoint is driven by the framework GAXI BFMs. TB classes live in `bin/TBClasses/cxs/`.

Test	DUT	Grid
`val/amba/test_cxs_master.py`	`cxs_master`	issues A-D, CXS-Lite, with and without link control and parity
`val/amba/test_cxs_slave.py`	`cxs_slave`	the same grid, each config also run with a 3-deep receive FIFO
`val/amba/test_cxs_master_cg.py`	`cxs_master_cg`	the same grid; must gate in STOP, never without link control
`val/amba/test_cxs_slave_cg.py`	`cxs_slave_cg`	the same grid; ACTIVEREQ must wake it
`val/amba/monitor-lite/test_cxs_{master,slave}_monlite.py`	the `_monlite` wrappers	the same grid; every Ocelus packet class checked

```
source env_python
cd val/amba
make clean-all && make 'run-cxs_*-func-parallel'
```

Each traffic phase makes one claim: streaming, credit starvation (the DUT must stall, and never send without a credit), a full idle credit pool (exactly the bound, never more), idle and hint-driven deactivation with every credit returned, no deactivation inside a packet, and parity-error injection. The suite kills six deliberate RTL mutations: sending without a credit, never returning credits, granting one past the pool, dropping ACK with credits outstanding, ignoring `CXSDATACHK`, and deactivating mid-packet.

1.10 References

- Arm. *AMBA CXS Protocol Specification*, ARM IHI 0079, Issue A (Mar 2018), B (Dec 2020), C (Dec 2023), D (Sep 2025).

developer.arm.com/documentation/ihi0079. Chapters 2-7 throughout: 2.1.2 credit rules, 2.2 properties (Table 2-2) and compatibility (Table 2-3), 3.2 check signals (Table 3-2), 4.1 packet placement, 4.3 control fields (Tables 4-4, 4-5, 4-7), chapter 5 link activation, 6.1 continuous delivery, chapter 7 CXS-Lite. Citation only; the PDFs are not redistributed here.

1.11 Navigation

- **Back to rtl-amba Index**
- **Back to Main Documentation Index**

2 CXS Master

Module: `cxs_master.sv` **Location:** `rtl/amba/cxs/` **Status:** New (2026-10-15)

2.1 Overview

`cxs_master` is a CXS transmitter endpoint, the CXS counterpart of `axis4_master`. Flits arrive on a valid/ready FUB port, wait in a skid buffer, and leave on the credited CXS link one per cycle while the transmitter holds a credit and the link is in RUN. One parameter set covers every issue of IHI 0079 (see CXS Modules).

2.1.1 Key Features

- Credit counter for up to `CXS_MAX_CREDIT` credits; a grant is usable the cycle after it arrives (2.1.2)
- Link activation and deactivation (`Explicit_Credit_Return`): raises `CXSACTIVEREQ` on demand, returns every credit on `CXSCRDRTN` before STOP, never deactivates mid-packet
- Deactivation on idleness (`cfg_idle_cycles`), on `CXSDEACTHINT`, or on `cfg_link_enable` low
- `Odd_Byte_Parity` check-signal generation, with receiver check signals verified every cycle
- All optional signals: `CXSCNTL`, `CXSLAST`, `CXSPRCLTYPE`, `CXSSRCID`, `CXSTGTID`, `CXSUSER`
- No output depends combinationally on an input
- Sticky `protocol_error` and `parity_error` status

2.2 Parameters

The `CXS_*` property parameters are described in CXS Modules: Properties and Parameters. An illegal combination for `CXS_ISSUE` stops elaboration with the rule number.

Parameter	Type	Default	Description
CXS_ISSUE	int	4 (D)	Spec issue the property set is checked against
CXS_DATA_WIDT	int	256	`CXSDATAFLITWIDTH`

Parameter	Type	Default	Description
H			
CXS_MAX_PKT_PER_FLIT	int	2	`CXSMAXPKTPERFLIT`; 1 = CXS-Lite (no `CXSCNTL`)
CXS_START_ALIGN	int	16	`CXS_START_ALIGNMENT` in bytes
CXS_HAS_LAST / CXS_HAS_PRCLTYPE	bit	0	`CXS_LAST` / `CXS_PROTOCOL_TYPE`
CXS_SRCID_WIDTH / CXS_TGTID_WIDTH / CXS_USER_WIDTH	int	0	Issue D sideband widths, 0 = absent
CXS_MAX_CREDIT	int	15	Credits this transmitter can track
CXS_CREDIT_BEYOND_SPEC	bit	0	Lift the 63-credit cap (rule 12) for a whole-SRAM pool; both ends must agree
CXS_STREAM_PRCL_BITS / CXS_STREAM_SRCID_BITS / CXS_STREAM_TGTID_BITS	int	0	Per-stream packet framing key (see Streams and Interleaving); 0/0/0 = one stream
CXS_LINK_CTRL	bit	1	1 = `Explicit_Credit_Return`
CXS_CHECK	bit	0	1 = `Odd_Byte_Parity`
SKID_DEPTH	int	4	FUB skid buffer depth, 2..8

2.2.1 Derived Values (do not override)

Name	Derivation	Meaning
CNTL_W	`cxs_pkg::cntl_width(N, DW, CXS_START_ALIGN)`	`CXSCNTLWIDTH`, 0 for CXS-Lite
CNTL_PW / SRCID_PW / TGTID_PW / USER_PW	max(width, 1)	Port widths of optional signals
DATACHK_W	DW/8	`CXSDATACHK` width
CNTLCHK_W / USERCHK_W	max(ceil(width/8), 1)	`CXSCNTLCHK` / `CXSUSERCHK` widths
CRW	clog2(CXS_MAX_CREDIT + 1)	Credit counter width

2.3 Ports

2.3.1 Clock, Reset and Configuration

Port	Width	Direction	Description
aclk	1	Input	Clock
aresetn	1	Input	Active-low asynchronous reset
cfg_link_enable	1	Input	0 = take the link down when idle and never raise it
cfg_link_hold	1	Input	1 = keep the link in RUN with nothing to send
cfg_idle_cycles	8	Input	Idle RUN cycles before deactivating; 0 = never on idleness alone

2.3.2 FUB Interface (valid/ready, input side)

Port	Width	Direction	Description
fub_cxs_valid / fub_cxs_ready	1	In / Out	Flit handshake (ready = skid not full)
fub_cxs_data	DW	Input	Flit data; byte n is `data[8n+7:8n]`
fub_cxs_cntl	CNTL_PW	Input	`CXSCNTL`, passed through unchanged
fub_cxs_last	1	Input	`CXSLAST`
fub_cxs_prcltype	3	Input	`CXSPRCLTYPE`
fub_cxs_srcid / fub_cxs_tgtid / fub_cxs_user	*_PW	Input	Issue D sidebands

2.3.3 CXS Transmitter Interface

Port	Width	Direction	Description
m_cxs_valid	1	Output	`CXSVALID`: a flit this cycle (spends a credit)
m_cxs_data, m_cxs_cntl, m_cxs_last, m_cxs_prcltype, m_cxs_srcid, m_cxs_tgtid, m_cxs_user	as FUB	Output	Flit payload; zero while `CXSVALID` is low
m_cxs_crdgnt	1	Input	`CXSCRDGNT`: one credit per cycle
m_cxs_crdrtn	1	Output	`CXSCRDRTN`: a credit returned (only while `CXSACTIVEREQ` is low)

Port	Width	Direction	Description
m_cxs_activereq	1	Output	CXSACTIVEREQ
m_cxs_activeack	1	Input	CXSACTIVEACK, sampled into a flop
m_cxs_deacthint	1	Input	CXSDEACTHINT, sampled into a flop

2.3.4 Check Signals (CXS_CHECK = 1)

Port	Width	Direction	Description
m_cxs_validchk	1	Output	~CXSVALID
m_cxs_datachk	DATACHK_W	Output	Odd parity, bit n covers data[8n+7:8n]
m_cxs_cntlchk	CNTLCHK_W	Output	Odd parity over CXSCNTL, short top byte
m_cxs_lastchk / m_cxs_prcltypechk / m_cxs_srcidchk / m_cxs_tgtidchk	1	Output	One bit each
m_cxs_userchk	USERCHK_W	Output	Odd parity over CXSUSER
m_cxs_crdrtnchk / m_cxs_activereqchk	1	Output	Inversions
m_cxs_crdgntchk / m_cxs_activeackchk	1	Input	Checked every cycle out of reset

2.3.5 Status

Port	Width	Direction	Description
busy	1	Output	Any FUB flit, skid entry, credit held, or link state other

Port	Width	Direction	Description
			than STOP
credits	CRW	Output	Credits held
link_state	2	Output	{ACTIVEREQ, sampled ACTIVEACK}: 00 STOP, 10 ACTIVATE, 11 RUN, 01 DEACTIVATE
in_packet	1	Output	A packet started on the link and not yet ended
protocol_error	1	Output	Sticky: a grant past `CXS_MAX_CREDIT`, ACK without REQ, or malformed `CXSCNTL` from the FUB
parity_error	1	Output	Sticky: `CXSCRDGNTCHK` or `CXSACTIVEACKCHK` mismatch

2.4 Functional Description

2.4.1 Credits

`r_credits` updates every cycle by `+CXSCRDGNT - CXSVALID - CXSCRDRTN`. A gain and a spend in the same cycle cancel. A flit goes out when the skid holds one, the link is in RUN, and `r_credits` is nonzero. `r_credits` is a register, so a grant cannot be spent in the cycle it arrives. A grant that would push the count past `CXS_MAX_CREDIT` is dropped and flagged in `protocol_error`.

2.4.2 Link control

From	To	When
STOP	ACTIVATE	`cfg_link_enable` and (a FUB flit, a skid entry, or `cfg_link_hold`)
ACTIVATE	RUN	the receiver raises `CXSACTIVEACK`
RUN	DEACTIVATE	skid empty, no packet open, and one of: `cfg_link_enable` low, `CXSDEACTHINT` with nothing to send, `cfg_idle_cycles` idle cycles
DEACTIVATE	STOP	the receiver drops `CXSACTIVEACK` after every credit is home

While `CXSACTIVEREQ` is low, `CXSCRDRTN` returns one held credit per cycle, including grants that arrive late. `CXSVALID` needs `CXSACTIVEREQ`, so the two are never high together.

2.4.3 Packet tracking

`cxs_cntl_track` counts the packets each sent flit starts and ends. That gives `in_packet`, which blocks deactivation, and format checks: thermometer `START`/`END`, `ENDERROR` only with `END`, and no packet ending before it started.

2.5 Timing Characteristics

- FUB to link: one skid buffer, so a flit accepted at cycle t can be sent at t+1 at the earliest.
- `CXSCRDGNT` at cycle t: usable at t+1.
- `CXSACTIVEACK` rising at cycle t: the first flit goes at t+1 at the earliest (registered sample).
- Back-to-back flits at one per cycle need `CXS_MAX_CREDIT` at least as large as the credit round trip (2.1.2).

2.6 Usage Example

```
cxs_master #(
    .CXS_ISSUE            (cxs_pkg::CXS_ISSUE_B),
    .CXS_DATA_WIDTH       (512),
    .CXS_MAX_PKT_PER_FLIT (2),
    .CXS_HAS_LAST         (1'b1),
    .CXS_MAX_CREDIT       (15),
    .CXS_LINK_CTRL        (1'b1),
    .CXS_CHECK            (1'b0)
) u_cxs_tx (
    .aclk            (aclk),
    .aresetn         (aresetn),
    .cfg_link_enable (1'b1),
    .cfg_link_hold   (1'b0),
    .cfg_idle_cycles (8'd32),
    .fub_cxs_valid   (flit_valid),   .fub_cxs_ready   (flit_ready),
    .fub_cxs_data    (flit_data),    .fub_cxs_cntl    (flit_cntl),
    .fub_cxs_last    (flit_last),    .fub_cxs_prcltype(3'b000),
    .fub_cxs_srcid   ('0), .fub_cxs_tgtid ('0), .fub_cxs_user ('0),
    .m_cxs_valid     (CXSTXVALID),   .m_cxs_data      (CXSTXDATA),
    .m_cxs_cntl      (CXSTXCNTL),    .m_cxs_last      (CXSTXLAST),
    .m_cxs_crdgnt    (CXSTXCRDGNT),  .m_cxs_crdrtn    (CXSTXCRDRTN),
    .m_cxs_activereq (CXSTXACTIVEREQ), .m_cxs_activeack
(CXSTXACTIVEACK),
    .m_cxs_deacthint (CXSTXDEACTHINT),
    .m_cxs_crdgntchk ('0), .m_cxs_activeackchk ('0),
    // remaining outputs left unconnected or monitored
    .busy (), .credits (), .link_state (), .in_packet (),
    .protocol_error (tx_proto_err), .parity_error ()
);
```

2.7 Design Notes

- **The FUB supplies `CXSCNTL`.** This block carries flits; it does not pack packets into them. The framework `CXSPacketizer` shows the placement rules.
- **CXS-Lite** (`CXS_MAX_PKT_PER_FLIT = 1`, Issue C/D): there is no `CXSCNTL`, every flit is one packet, and `in_packet` is always 0.
- **Check signals are generated, not carried.** Protection starts at the interface, so the payload `*CHK` bits are computed from the outgoing flit. That differs from `axis5_master`, which carries the FUB's check bits end to end.

2.8 Related Modules

- **CXS Slave**: the receiver
- **CXS Master CG**: clock-gated variant
- **AXIS4 Master**: the valid/ready endpoint this one mirrors
- `gaxi_skid_buffer`, `cxs_cntl_track`, `cxs_odd_byte_parity`, `cxs_pkg`

2.9 Testing

`val/amba/test_cxs_master.py` uses a GAXI master on `fub_cxs_*`, the framework `CXSReceiver` on `m_cxs_*`, and an independent `CXSMonitor`. The phases are fast, starve, gappy, back-to-back, idle pool fill, idle deactivation, reactivation, hint deactivation, mid-packet idle and parity injection.

```
source env_python
cd val/amba && make clean-all && make run-cxs_master-func-parallel
```

2.10 References

- Arm. *AMBA CXS Protocol Specification*, ARM IHI 0079D, Sep. 2025 (and Issues A-C). developer.arm.com/documentation/ihi0079/d: 2.1.1 clock and reset, 2.1.2 credit exchange, 3.2 check signals, 5.2-5.6 link activation, 6.1 continuous delivery. Citation only.

2.11 Navigation

- **Back to CXS Index**
- **Back to rtl-amba Index**
- **Back to Main Documentation Index**

3 CXS Master with Clock Gating

Module: `cxs_master_cg.sv` **Location:** `rtl/amba/cxs/` **Status:** New (2026-10-15)

3.1 Overview

`cxs_master_cg` puts cxs_master behind one `amba_clock_gate_ctrl`. It has the same parameters and ports as the core, minus `busy`, plus the clock-gating set: `CG_IDLE_COUNT_WIDTH`, `cfg_cg_enable`, `cfg_cg_idle_count`, `cg_gating` and `cg_idle`.

3.2 When It Gates

A credited link changes state with no handshake the gated side could stall. A `CXSCRDGNT` that arrives while the clock is stopped is a credit lost for good. The core's `busy` therefore covers every state in which the receiver may still grant: `CXSACTIVEREQ` or the sampled `CXSACTIVEACK` high, or any credit held. As a result:

CXS_LINK_CTRL	Gates?	Why
1	Only in STOP with zero credits	The receiver "must not send credit" in STOP (IHI 0079D 5.2). A FUB flit wakes the block, which then raises `CXSACTIVEREQ`
0	Never	The receiver grants whenever it has room, so the core holds credits indefinitely and `busy` never falls

The second row is a property of the protocol, not a shortcoming of this wrapper. To save clock power, use link control.

3.3 Activity Terms

The terms follow `vault/handbook/design/clock-gating-activity-terms.md`:

Term	Expression	Reason
user_valid	`fub_cxs_valid | busy`	FUB VALID plus all pending work

Term	Expression	Reason
axi_valid	`m_cxs_crdgnt | m_cxs_activeack`	The peer's VALID-like inputs, synchronous to `aclk` (5.5)

No READY appears in either term; CXS has none. The event outputs `CXSVALID` and `CXSCRDRTN` are masked with `!cg_gating` for the wake-latency overlap, and their check bits follow the masked value (a masked 0 carries check 1). `fub_cxs_ready` is forced low while gated.

3.4 Testing

`val/amba/test_cxs_master_cg.py` runs the master traffic with `cfg_cg_enable = 1` and checks every flit. It then makes two gating claims: with link control the block **must** gate during an idle window in STOP, and without link control it must **never** gate.

3.5 Navigation

- **Back to CXS Index**
- **CXS Master**

4 CXS Slave

Module: `cxs_slave.sv` **Location:** `rtl/amba/cxs/` **Status:** New (2026-10-15)

4.1 Overview

`cxs_slave` is a CXS receiver endpoint, the CXS counterpart of `axis4_slave`. Flits arrive on the credited link, land in a receive FIFO, and leave on a valid/ready FUB port. The receive FIFO **is** the credit pool. A credit is granted only when a FIFO slot is free to back it, so the receiver never refuses a flit. That is the guarantee 2.1.2 requires ("the Receiver must guarantee that it can receive one flit of data for each credit that it grants").

4.1.1 Key Features

- Credit grant bounded by FIFO room and `CXS_MAX_CREDIT`; a spent or returned credit is reissued the next cycle at the earliest
- `RX_FIFO_DEPTH` may be smaller than `CXS_MAX_CREDIT`, in which case the FIFO sets the pool size
- Link control: synchronizes the asynchronous `CXSACTIVEREQ`, raises `CXSACTIVEACK`, grants only in RUN, and drops ACK only once every credit is home
- `CXSDEACTHINT` driven from `cfg_deact_hint`
- `Odd_Byte_Parity` checking of every received signal, with its own check signals generated
- Sticky `protocol_error`: a flit or return with no credit, `CXSVALID` together with `CXSCRDRTN`, FIFO overflow, malformed `CXSCNTL`

4.2 Parameters

The `CXS_*` property parameters are the same as for cxs_master. Two more:

Parameter	Type	Default	Description
RX_FIFO_DEPTH	int	CXS_MAX_CREDIT	Receive FIFO depth, i.e. the credit pool. Any depth >= 1. Below the credit round trip it throttles the

Parameter	Type	Default	Description
			link
CXS_CREDIT_BEYOND_SPEC	bit	0	Lift the 63-credit cap (rule 12) for a whole-SRAM pool; both ends must agree
CXS_STREAM_PRCL_BITS / CXS_STREAM_SRCID_BITS / CXS_STREAM_TGTID_BITS	int	0	Per-stream packet framing key (see Streams and Interleaving); 0/0/0 = one stream
ACTIVEREQ_SYNC_STAGES	int	2	Synchronizer flops on `CXSACTIVEREQ` (and its check bit); 0 = known-synchronous system

Derived values (`CNTL_W`, the `*_PW` port widths, the check widths and `CRW`) are as for `cxs_master`.

4.3 Ports

4.3.1 Clock, Reset and Configuration

Port	Width	Direction	Description
aclk / aresetn	1	Input	Clock, active-low asynchronous reset
cfg_rx_enable	1	Input	0 = never activate, never grant
cfg_deact_hint	1	Input	Drives

Port	Width	Direction	Description
			CXSDEACTHINT (registered)

4.3.2 CXS Receiver Interface

Port	Width	Direction	Description
s_cxs_valid	1	Input	CXSVALID
s_cxs_data, s_cxs_cntl, s_cxs_last, s_cxs_prcltype, s_cxs_srcid, s_cxs_tgtid, s_cxs_user	see cxs_master	Input	Flit payload; absent signals ignored
s_cxs_crdgnt	1	Output	CXSCRDGNT, registered
s_cxs_crdrtn	1	Input	CXSCRDRTN
s_cxs_activereq	1	Input	CXSACTIVEREQ, asynchronous, synchronized here
s_cxs_activeack	1	Output	CXSACTIVEACK
s_cxs_deacthint	1	Output	CXSDEACTHINT

4.3.3 Check Signals (CXS_CHECK = 1)

The s_cxs_*chk inputs mirror the m_cxs_*chk outputs of cxs_master. s_cxs_crdgntchk and s_cxs_activeackchk are generated outputs.

4.3.4 FUB Interface (valid/ready, output side)

Port	Width	Direction	Description
fub_cxs_valid / fub_cxs_ready	1	Out / In	FIFO not empty / consumer ready
fub_cxs_data … fub_cxs_user	as received	Output	The flit, absent fields zero

4.3.5 Status

Port	Width	Direction	Description
busy	1	Output	A flit arriving or queued, credits out, or link state other than STOP
credits_outstanding	CRW	Output	Granted and not yet spent or returned
link_state	2	Output	{synchronized ACTIVEREQ, ACTIVEACK}
in_packet	1	Output	A packet started and not yet ended
protocol_error / parity_error	1	Output	Sticky error flags

4.4 Functional Description

4.4.1 Credit pool

`r_out` counts credits outstanding, including the grant on the wire this cycle. Each cycle the block works out the state after the clock edge, before any new grant:

```
out_next = r_out + CXSCRDGNT(now) - CXSVALID - CXSCRDRTN
occ_next = fifo_count + CXSVALID - fub_pop
grant    = RUN && out_next < CXS_MAX_CREDIT && occ_next + out_next < RX_FIFO_DEPTH
```

`r_crdgnt <= grant`, so `CXSCRDGNT` is a flop fed from flops and this cycle's handshake. A credit consumed at cycle t is reissued at t+1 at the earliest.

`gaxi_fifo_sync` cannot be built 1 deep, but one credit with a one-flit buffer is a legal receiver. The FIFO is therefore built at least 2 deep while the grant still uses `RX_FIFO_DEPTH`, so the spare slot is never granted against.

4.4.2 Link control

`CXSACTIVEREQ` and `CXSACTIVEREQCHK` cross through one `glitch_free_n_dff_arn`. The check bit crosses **inverted**: the synchronizer flops reset to 0, and the correct check for a reset-low

ACTIVEREQ is 1. Carried uninverted, the pair would read `{0,0}`, a parity error, for every cycle the synchronizer is flushing after reset. The test suite caught exactly that.

- ACTIVATE to RUN: `CXSACTIVEACK` rises when the synchronized ACTIVEREQ is high and `cfg_rx_enable` is set.
- Credits are granted only while both are high.
- DEACTIVATE: flits still in flight are accepted, and ACK falls only when `out_next` is 0.

4.5 Timing Characteristics

- Link to FUB: the FIFO is in mux-read mode, so a flit received at cycle t is offered at t+1.
- Credit reuse: a slot freed at cycle t (FUB pop) supports a grant driven at t+1.
- ACTIVEREQ to ACTIVEACK: `ACTIVEREQ_SYNC_STAGES` + 1 cycles; the first grant follows one cycle after ACK.

4.6 Usage Example

```
cxs_slave #(
    .CXS_ISSUE            (cxs_pkg::CXS_ISSUE_D),
    .CXS_DATA_WIDTH       (256),
    .CXS_MAX_PKT_PER_FLIT (3),
    .CXS_START_ALIGN      (4),
    .CXS_SRCID_WIDTH      (4),
    .CXS_TGTID_WIDTH      (2),
    .CXS_MAX_CREDIT       (15),
    .RX_FIFO_DEPTH        (8)
) u_cxs_rx (
    .aclk (aclk), .aresetn (aresetn),
    .cfg_rx_enable (1'b1), .cfg_deact_hint (want_idle),
    .s_cxs_valid (CXSRXVALID), .s_cxs_data (CXSRXDATA), .s_cxs_cntl
(CXSRXCNTL),
    .s_cxs_last ('0), .s_cxs_prcltype ('0),
    .s_cxs_srcid (CXSRXSRCID), .s_cxs_tgtid (CXSRXTGTID), .s_cxs_user
('0),
    .s_cxs_crdgnt (CXSRXCRDGNT), .s_cxs_crdrtn (CXSRXCRDRTN),
    .s_cxs_activereq (CXSRXACTIVEREQ), .s_cxs_activeack
(CXSRXACTIVEACK),
    .s_cxs_deacthint (CXSRXDEACTHINT),
    // check inputs tied when CXS_CHECK = 0
    .s_cxs_validchk ('0), .s_cxs_datachk ('0), .s_cxs_cntlchk
```

```
('0), .s_cxs_lastchk ('0),
    .s_cxs_prcltypechk ('0), .s_cxs_srcidchk ('0), .s_cxs_tgtidchk
('0), .s_cxs_userchk ('0),
    .s_cxs_crdrtnchk ('0), .s_cxs_activereqchk ('0),
    .s_cxs_crdgntchk (), .s_cxs_activeackchk (),
    .fub_cxs_valid (rx_valid), .fub_cxs_ready
(rx_ready), .fub_cxs_data (rx_data),
    .fub_cxs_cntl (rx_cntl), .fub_cxs_last (), .fub_cxs_prcltype (),
    .fub_cxs_srcid (rx_src), .fub_cxs_tgtid (rx_tgt), .fub_cxs_user
(),
    .busy (), .credits_outstanding (), .link_state (), .in_packet (),
    .protocol_error (rx_proto_err), .parity_error ()
);
```

4.7 Related Modules

- **CXS Master**: the transmitter
- **CXS Slave CG**: clock-gated variant
- **AXIS4 Slave**: the valid/ready endpoint this one mirrors
- `gaxi_fifo_sync`, `glitch_free_n_dff_arn`, `cxs_cntl_track`, `cxs_odd_byte_parity`

4.8 Testing

`val/amba/test_cxs_slave.py` uses the framework `CXSTransmitter` on `s_cxs_*`, a GAXI slave on `fub_cxs_*`, and a `CXSMonitor`. Every config runs twice, once with `RX_FIFO_DEPTH = CXS_MAX_CREDIT` and once with a 3-deep FIFO. The idle pool-fill phase requires exactly `min(RX_FIFO_DEPTH, CXS_MAX_CREDIT)` credits at the transmitter.

```
source env_python
cd val/amba && make clean-all && make run-cxs_slave-func-parallel
```

4.9 References

- Arm. *AMBA CXS Protocol Specification*, ARM IHI 0079D, Sep. 2025 (and Issues A-C). developer.arm.com/documentation/ihi0079/d: 2.1.2 credit rules, 5.2 receiver obligations in each state, 5.4 race conditions, 5.5 asynchronous `CXSACTIVEREQ`. Citation only.

4.10 Navigation

- **Back to CXS Index**
- **Back to rtl-amba Index**
- **Back to Main Documentation Index**

5 CXS Slave with Clock Gating

Module: `cxs_slave_cg.sv` **Location:** `rtl/amba/cxs/` **Status:** New (2026-10-15)

5.1 Overview

`cxs_slave_cg` puts cxs_slave behind one `amba_clock_gate_ctrl`. It has the same parameters and ports as the core, minus `busy`, plus `CG_IDLE_COUNT_WIDTH`, `cfg_cg_enable`, `cfg_cg_idle_count`, `cg_gating` and `cg_idle`.

IHI 0079D 5.5 anticipates this block: “It is common for the clock in the Receiver to stop during the STOP state … The assertion of the CXSACTIVEREQ signal might be used to restart that clock.”

5.2 When It Gates

The core’s `busy` covers every state in which a flit or credit return can still arrive: ACK or the synchronized ACTIVEREQ high, any credit outstanding, or a flit waiting on the FUB side.

CXS_LINK_CTRL	Gates?	Why
1	Only in STOP, every credit home, FIFO empty	The transmitter “must not send flits or credit returns” in STOP (5.2); a rising `CXSACTIVEREQ` wakes the block
0	Never	Credits stay outstanding indefinitely, so a flit can arrive on any cycle

5.3 Activity Terms

Term	Expression	Reason
user_valid	`busy`	Includes `fub_cxs_valid`, the pending output
axi_valid	`s_cxs_valid | s_cxs_crdrtn | ACTIVEREQ (synchronized)`	Peer VALIDs and the wake request

`CXSACTIVEREQ` is asynchronous, so the wake term passes through its own `glitch_free_n_dff_arn` on the **free-running** `aclk`. A wake term must be in the gate’s own

clock domain, and the core's synchronizer sits on the gated clock and is frozen in STOP. `CXSCRDGNT` (and its check bit) and `fub_cxs_valid` are masked with `!cg_gating`.

5.4 Testing

`val/amba/test_cxs_slave_cg.py` streams flits, takes the link to STOP, idles, and requires the block to have gated during that window. It then sends a wake phase that must be delivered intact, which proves `CXSACTIVEREQ` restarted the clock. Without link control it must never gate.

5.5 Navigation

- **Back to CXS Index**
- **CXS Slave**